

TIPO DE ESPACIO AL QUE CORRESPONDE EL PLAN		TRONCAL
NOMBRE DEL ESPACIO		SOFTWARE
NOMBRE DEL PLAN:		"Tu app pide una pizza... y la API se la entrega"
DURACIÓN	1 BIMESTRE	UBICACIÓN TEMPORAL DEL PLAN 2DO BIMESTRE
SINOPSIS: <i>En este desafío, tus líneas de código van a cobrar hambre: vas a desarrollar una aplicación distribuida en C# donde el backend simula una pizzería digital que recibe pedidos a través de una API REST. A partir de modelos de casos de uso, definirás cómo interactúan los distintos actores del sistema: desde el cliente hambriento hasta la cocina automatizada. Vas a implementar programación distribuida usando sockets para coordinar procesos entre módulos independientes, y te enfrentarás al tratamiento de errores reales en entornos asíncronos, donde una excepción puede arruinar la entrega (o la pizza). Este proyecto es una experiencia completa que une teoría y práctica para que entiendas, desde adentro, cómo fluye la información en aplicaciones modernas conectadas en red.</i>		
CONTENIDOS: <i>A lo largo de este plan aprenderás:</i> <i>Definición Backend, uso y consumo de Api Rest, modelos de caso de uso, diseño de casos de uso, jerárquico, delegación. Programación distribuida, sockets. Manipulación de errores en la programación asíncrona y distribuida (excepciones)</i>		
OBJETIVOS: <i>Se espera que logres: ... (objetivos, en infinitivo).</i> <i>Desarrollar y consumir una API REST, integrando esta arquitectura nueva con la Programación Concurrente ya adquirida en el bimestre anterior.</i>		

PUNTO DE PARTIDA

En esta etapa vas a descubrir el desafío: construir una app distribuida que simule el pedido y entrega de una pizza. A partir de una situación cotidiana, vas a identificar qué componentes necesita un sistema como este, cómo se comunican entre sí y qué tipo de problemas técnicos podrían aparecer.

DURACIÓN ESTIMADA DE LA ETAPA - 2 SEMANAS

RECURSOS

Computadora con conectividad a internet.

Acceso a recursos en línea para la realización de las actividades.

Editor de código y documentación.

ACTIVIDADES

Actividad 1: Programación Distribuida

Introducción a la programación distribuida, repasando el modelo cliente-servidor, comparación con otros modelos. Llevando nuestra aplicación local a distribuida. Presentando el caso de uso de la pizzeria.

Actividad 2: Identificación de los componentes del sistema

Análisis del relevamiento, diseño de clases, diagramas de secuencia

Tengan en cuenta que las mínimas entidades son:

- Pizza
- Cliente (sin contraseña)
- Pedido, destacando que sus 3 posibles estados son:
 - Espera de confirmación
 - En preparación
 - En viaje
 - Entregado

Actividad 3: Desafíos de nuestra aplicación

Clasificación de los posibles fallos que podamos encontrar y elección de acciones ante estos tipos de imponderables.



INDAGACIÓN

Ahora es momento de pensar cómo resolver el problema: vas a diseñar los casos de uso del sistema, identificar los actores y sus responsabilidades, y comprender cómo se conectan mediante APIs y sockets. También vas a investigar cómo manejar errores en programación distribuida y asíncrona, preparando todo lo necesario para pasar al código.

DURACIÓN ESTIMADA DE LA ETAPA - 3 SEMANAS

RECURSOS

Computadora con conectividad a internet.
Acceso a recursos en línea para la realización de las actividades.
Editor de código y documentación.

ACTIVIDADES

Actividad 1: Identificación de actores y elaboración de casos de uso

Analizar el sistema propuesto para reconocer los distintos actores (usuario, backend, cocina, etc.) y describir sus interacciones mediante modelos de caso de uso.

Actividad 2: Diseño jerárquico y delegación de responsabilidades

Organizar los casos de uso en un esquema jerárquico que refleje cómo se delegan tareas entre componentes del sistema, diferenciando claramente los roles del cliente, API y servicios internos.

Actividad 3: Esquema de arquitectura distribuida del sistema

Dibujar un esquema o diagrama que represente cómo se distribuyen los módulos, qué tipo de comunicación usan (API REST, sockets), y qué mensajes o datos intercambian.

Actividad 4: Exploración práctica de APIs REST

Usar herramientas como Postman o Swagger para consultar y consumir APIs públicas o simuladas, con el objetivo de entender cómo se estructuran los endpoints, respuestas y errores.

Actividad 5: Análisis de manejo de errores asíncronos en C#

Estudiar fragmentos de código con *async/await*, *try/catch* y fallos simulados en red o tareas concurrentes. Identificar buenas prácticas para controlar errores en sistemas distribuidos..

PRODUCCIÓN

Llegó la hora de construir. Vas a programar una solución funcional con cliente, backend y servicios conectados por red. Vas a implementar comunicación con APIs REST y sockets, vas a aplicar lo que sabes sobre asincronía y manejo de errores, para probar tu sistema en condiciones reales, asegurándote de que cada pedido llegue... caliente y sin excepciones.

DURACIÓN ESTIMADA DE LA ETAPA - 3 SEMANAS

RECURSOS

Computadora con conectividad a internet.

Acceso a recursos en línea para la realización de las actividades.

Editor de código y documentación.

ACTIVIDADES

Actividad 1: Implementación del cliente que realiza el pedido

Programar una aplicación en C# que simule al usuario final haciendo pedidos de pizza a través de una API REST. El cliente debe enviar datos estructurados y recibir confirmaciones.

Actividad 2: Desarrollo del backend con API REST y lógica de negocio

Crear un backend en C# con Minimal APIs que exponga los endpoints necesarios para recibir y procesar los pedidos. Este módulo debe gestionar el pedido, registrar la información y delegar tareas internas.

Actividad 3: Integración de servicios internos mediante sockets

Implementar componentes que simulen áreas internas del negocio (como la cocina o el reparto), comunicándose con el backend a través de sockets. Estos servicios responderán a tareas específicas delegadas desde el backend.

Actividad 4: Manejo de asincronía y errores distribuidos

Asegurar que todas las comunicaciones (cliente al API, API a servicios internos) se manejen de forma asincrónica. Simular errores en la red o fallos internos (por ejemplo, caída del servicio de cocina) y aplicar buenas prácticas para detectarlos y tratarlos correctamente (logs, reintentos, mensajes de error amigables).

Actividad 5: Pruebas del sistema y presentación funcional

Ejecutar pruebas controladas que muestren el flujo completo de un pedido de pizza. Registrar y analizar qué ocurre cuando todo funciona bien... y qué pasa cuando algo falla. Presentar el proyecto como una demo funcional, destacando la arquitectura, el manejo de errores y las decisiones de diseño.

EVALUACIÓN

En esta etapa final vas a detenerte a reflexionar sobre lo que aprendiste a lo largo del proyecto. Vas a revisar cómo aplicaste los conceptos básicos de una API REST ampliando las capacidades de tus proyectos.

DURACIÓN ESTIMADA DE LA ETAPA - 1 SEMANA

RECURSOS

Computadora con conectividad a internet.

Acceso a recursos en línea para la realización de las actividades.

Editor de código y documentación.

ACTIVIDADES

- Presentación de producto terminado con su correspondiente documentación técnica
- Exposición oral comentando los procesos que intervinieron, haciendo foco en los desafíos que encontraron, como los pudieron sortear y que alternativas plantearon.
- Valoración del lenguaje técnico oral y en redacción.
- En caso de no haber concluido, evaluación de un informe técnico donde se expliquen las dificultades encontradas, a fin de responder para la rectificación del tp y conclusión del mismo a posterior.